

GitHub Copilot Architecture

Understanding Customization Layers: Agents, Skills, and Instructions

REFERENCE GUIDE & DEVELOPER SHEET

Executive Summary & Comparison Matrix

GitHub Copilot provides a three-tier architecture to customize model interaction: **Agents** (identity and orchestration), **Skills** (task-specific procedural playbooks), and **Instructions** (global behavioral rules and standards).

ASPECT	AGENTS	SKILLS	INSTRUCTIONS
Primary Focus	Identity & Tool Orchestration	Task Execution Playbooks	Behavioral Guardrails
Analogy	Specialized Team Member	Standard Operating Procedure (SOP)	Company Coding Style Guide
Activation	Explicit user selection or sub-agent delegation	On-demand: Contextually loaded when prompt matches skill description	Always-on or triggered by file matching globs
Scope	Cross-tool orchestration, tool definitions, persona	Modular, repeatable task workflows & templates	Repository-wide rules, architecture, formatting
Location	<code>.github/agents/</code>	<code>.github/skills/*/SKILL.md</code>	<code>.github/copilot-instructions.md</code> or <code>.github/instructions/*.md</code>

Deep Dive into Architecture Layers

1. Agents — "Who is doing the work?" Identity & Tools

An **Agent** represents a specialized persona configured with a specific toolset, system role, and scope of operations. Agents manage high-level decision-making and orchestrate tools or sub-agents.

- **How it works:** Configured to access Model Context Protocol (MCP) tools, shell operations, or specialized system prompts.
- **When to use:** Dedicated tasks requiring discrete tools (e.g., a *Database Migration Agent* with SQL connections or a *DevOps Release Specialist*).

2. Skills — "How to complete a specific task?"

[On-Demand Playbooks](#)

A **Skill** is a modular, self-contained package following the Agent Skills specification. It provides step-by-step guidance, references, templates, and scripts for specific repeatable tasks.

- **How it works:** To conserve context window memory, Copilot loads only the skill's name and description. When a user prompt triggers the skill's description, the full `SKILL.md` instructions load dynamically.
- **When to use:** Complex workflows that shouldn't pollute global context (e.g., generating API boilerplate, running migrations, or writing E2E tests).

3. Instructions — "What rules must always be followed?"

[Persistent Guardrails](#)

Instructions are persistent rules injected directly into Copilot's prompt context to enforce ongoing coding standards, security constraints, and architectural patterns.

- **How it works:** Always active in context, or selectively attached using file globs (e.g., `applyTo: "**/*.ts"`).
- **When to use:** Repository-wide rules like mandatory frameworks, security requirements, error-handling conventions, or naming structures.

How They Work Together

1. **Instructions** enforce baseline constraints (e.g., "All API endpoints must return RFC 7807 error JSON").
2. **Skills** supply procedural workflows on demand (e.g., "Execute this 5-step checklist and template when creating a new endpoint").
3. **Agents** execute the work by uniting instructions, executing skills, and invoking system tools (e.g., "API Engineer Agent").